

ESP32 Thin Linux Compatibility Layer

A Unikernel Approach to Dynamic ELF Execution on No-MMU Hardware

Dhruv Joshi (2022EE32079) Gourab Raj Sabat (2022EE11675)

Department of Electrical Engineering
Indian Institute of Technology Delhi

BTP Fall 2025
Supervisor: Prof. Kaushik Saha

The Design Problem

Goal: Execute standard POSIX C applications on ESP32 *without reflashing firmware*.

Why is this hard?

- ESP32 has **no MMU** — no virtual memory
- Firmware is **statically linked**
- 50-line app change → reflash 4MB
- No dynamic linker (ld.so)

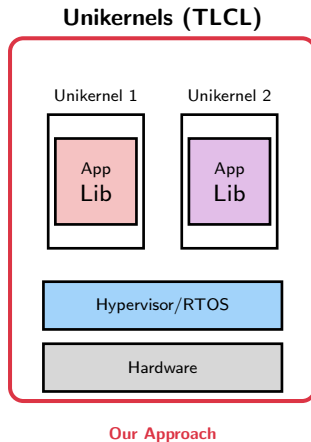
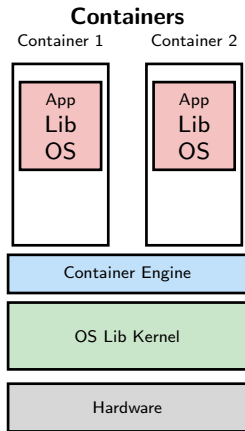
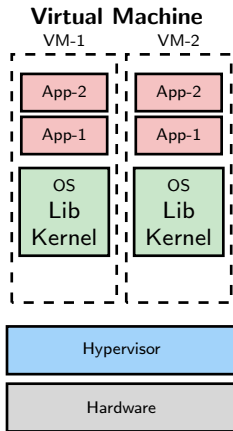
Approach: Library OS

- Linux *API* without Linux *architecture*
- Runtime ELF loading + symbol resolution
- Syscall translation layer (shims)
- Unix-style /dev/ abstraction

Result

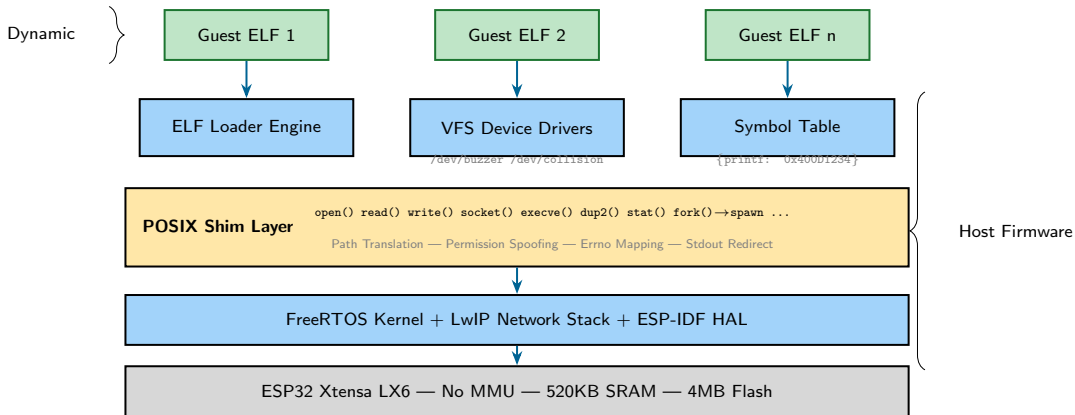
Standard C applications run *unmodified*, loaded at runtime over WiFi.

Where TLCL Fits: OS Architecture Taxonomy



Key Insight: TLCL links OS functionality *into* the application runtime — syscalls become function calls, not context switches.

System Architecture Overview



Memory Architecture: The No-MMU Constraint

Harvard Architecture Split:

IRAM (Instruction RAM)

0x40080000 - 0x400A0000
128KB — Execute-only bus

Code segments go here

DRAM (Data RAM)

0x3FFB0000 - 0x3FFFFFFF
320KB — Read/Write

Data, BSS, heap, stacks

The Address Space Problem

Linux processes expect fixed virtual addresses (e.g., 0x8048000). On ESP32, that address doesn't exist or maps to peripherals.

Solution: Position Independent Code

- Compile with `-fPIC -shared`
- All references use GOT/relative addressing
- Loader performs load-time relocation
- Addresses adjusted to actual malloc location

ELF Loader: Dynamic Linking on Bare Metal



Header Validation:

- Magic: 0x7F 'E' 'L' 'F'
- Class: ELFCLASS32, Machine: EM_XTENSA
- Type: ET_DYN (position independent)

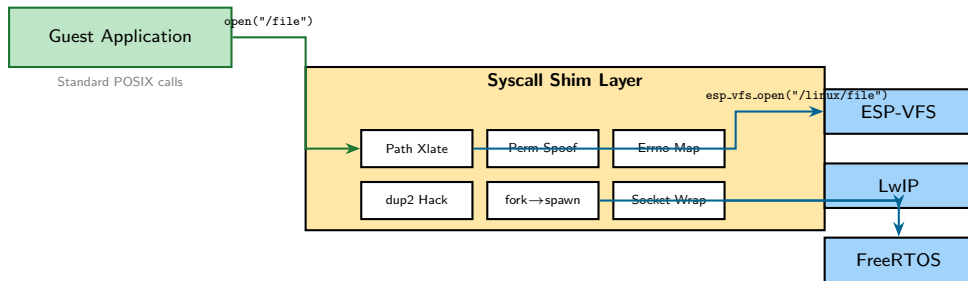
Why ET_DYN? Can load at any address.

Xtensa Relocations:

- R_XTENSA_RELATIVE: `*addr += load_bias`
- R_XTENSA_SLOT0_OP: Patch 18-bit opcode offset

Cache Coherency: Flush D-Cache, invalidate I-Cache.

POSIX Shim Layer: Syscall Translation



Path Translation

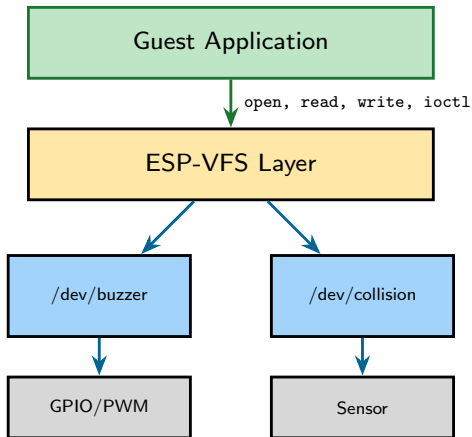
- `/file` → `/linux/file`
- `/dev/*` passed through
- Relative paths use task-local CWD

Permission Spoofing

- LittleFS stores no permissions
- `stat()` returns `st_mode = 0`
- Shim forces 0777 for compatibility

Unix-Style Hardware Abstraction: /dev/ Drivers

Design Goal: Standard POSIX applications control real hardware via familiar /dev/ interface.



Driver Interface:

- `open("/dev/buzzer", O_RDWR)`
- `write(fd, "1", 1)` — enable
- `ioctl(fd, SET_FREQ, 880)`
- `read(fd, buf, len)` — blocking

Implementation:

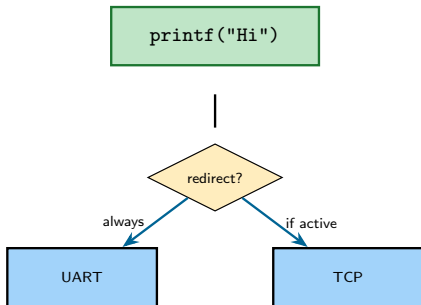
- `esp_vfs_register()` registration
- FreeRTOS queues for blocking I/O
- `ioctl()` for configuration

Key Benefit

stdout Redirection: The dup2 Challenge

The Problem:

- FD 0/1/2 hardwired to UART
- `close(1)` destabilizes logging
- Need output over TCP



Solution: Global State Interceptor

`shim_dup2(sock, STDOUT):`

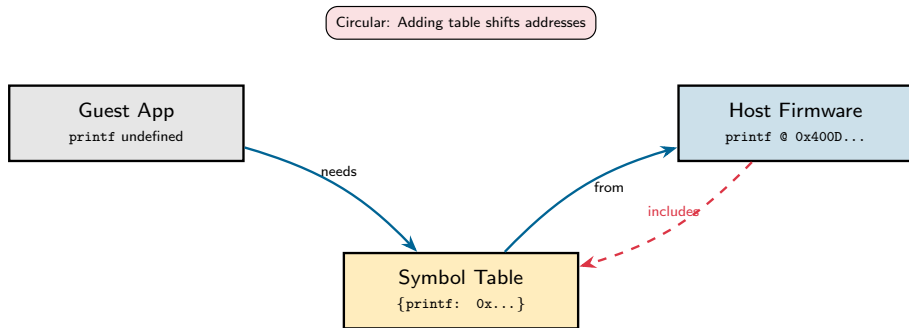
- Store socket FD in global state
- Don't touch VFS table

`shim_write(STDOUT, buf, len):`

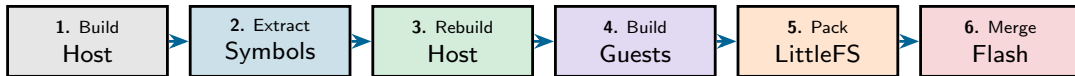
- Write to UART (debug)
- Send to socket (`MSG_DONTWAIT`)

Result: Dual-route output.

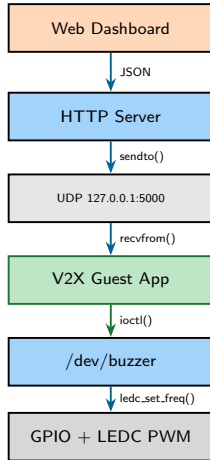
Two-Pass Build System: Resolving Circular Dependencies



Solution: Multi-Stage Pipeline



Demo 1: V2X Collision Avoidance System



Time-to-Collision Calculation:

$$TTC = \frac{|\vec{P}_{target} - \vec{P}_{ego}|}{|\vec{V}_{target} - \vec{V}_{ego}|}$$

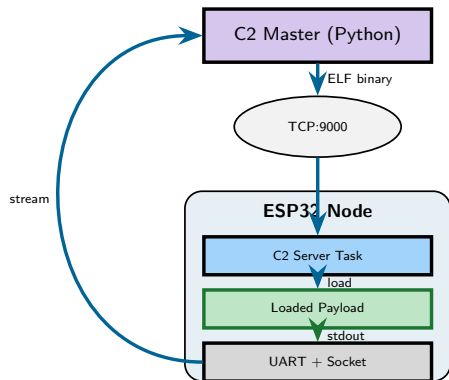
Application Logic

Open device: `open("/dev/buzzer", O_RDWR)`

If collision imminent:

- Set frequency: `ioctl(fd, SET_FREQ, 880)`
- Enable alarm: `write(fd, "1", 1)`

Demo 2: Distributed Command & Control



Protocol:

- 1 Master sends: 4-byte magic + 4-byte size + ELF
- 2 ESP32 writes to `/linux/payload.elf`
- 3 Loader parses, allocates, relocates, executes
- 4 stdout streams back over same connection

Advantage

Update 4KB application vs. reflash 4MB firmware.
50 drones: 100+ min → seconds.

Security Model & Limitations

Library OS Risks

- **No memory isolation:** Guest can access any physical address
- **No fault isolation:** Guest crash → full chip reboot
- **Shared address space:** Guests can interfere with each other

Resource Constraints:

Resource	Limit
Total DRAM	320KB
Available for guests	~100KB
Task stack (min)	4KB
Max concurrent guests	~15
Guest IRAM	~32KB

Threat Model

Trusted guest code in controlled deployments. Not suitable for public app stores.

Contributions & Future Directions

Key Contributions:

- ❶ **Dynamic ELF Loading**
 - Full Xtensa relocation support
 - IRAM/DRAM split allocation
- ❷ **POSIX Compatibility Layer**
 - File I/O with path translation
 - BSD sockets via LwIP
- ❸ **Unix-style Driver Model**
 - /dev/ nodes for hardware
 - ioctl() configuration

Future Directions:

- **Short term:**
 - Shared libraries (libc.so)
 - Multi-core pinning
- **Long term:**
 - MicroPython/Lua as guests
 - WASM for sandboxed execution
 - ESP32-S3 PMS integration

Vision

A microcontroller that behaves like a tiny Linux box —
dynamic, network-programmable, yet real-time.